

GUÍA PRÁCTICA 3

INTEGRACIÓN DE REACT CON LA API FASTAPI

Programación Web / Desarrollo Full Stack
Nivel: Intermedio • Individual / Parejas • 3 horas

Proyecto: Sistema web de gestión de usuarios

Frontend: React + Vite

Backend: FastAPI + SQLAlchemy

Base de datos: PostgreSQL

Diseño visual inspirado en la identidad de SENA

1. Presentación de la actividad

En las dos primeras guías se construyó el Backend del proyecto: primero una API básica con FastAPI y posteriormente una API conectada a PostgreSQL mediante SQLAlchemy y organizada por capas. En esta tercera guía se construirá el Frontend con React y se conectará al Backend mediante solicitudes HTTP.

El resultado será una aplicación web moderna para consultar, registrar, buscar y eliminar usuarios. El aprendiz desarrollará la interfaz en Visual Studio Code y aprenderá a separar claramente Frontend y Backend.

2. Objetivo general

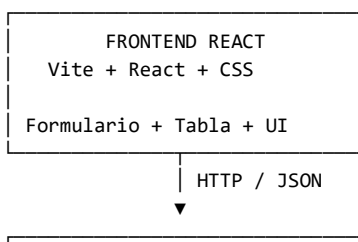
Desarrollar una interfaz web con React que consuma la API de usuarios creada con FastAPI, permitiendo administrar los registros almacenados en PostgreSQL mediante una interfaz moderna, responsiva y fácil de utilizar.

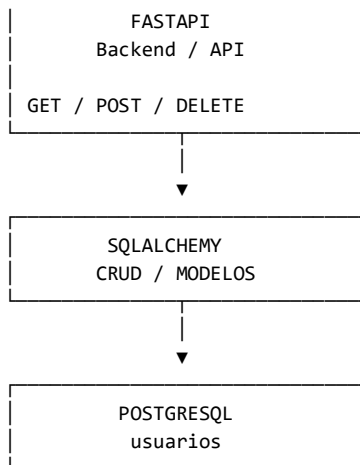
3. Objetivos específicos

- Instalar y configurar Node.js y Visual Studio Code.
- Crear un proyecto React utilizando Vite.
- Comprender la estructura básica de un proyecto React.
- Crear componentes reutilizables.
- Utilizar useState y useEffect.
- Consumir una API REST utilizando fetch.
- Mostrar usuarios obtenidos desde FastAPI.
- Crear un formulario para registrar usuarios.
- Eliminar usuarios desde React.
- Mostrar mensajes de éxito y error.
- Crear una interfaz visual moderna inspirada en los colores del SENA.
- Comprender la comunicación React → FastAPI → SQLAlchemy → PostgreSQL.

4. Arquitectura completa del proyecto

Al finalizar la Guía 3, el sistema tendrá tres grandes bloques:





5. ¿Qué responsabilidad tiene React?

React se encarga de la presentación y de la interacción con el usuario. React NO debe conectarse directamente a PostgreSQL. La comunicación con la base de datos debe realizarse a través del Backend.

React sí hace	React no hace	Motivo
Mostrar formularios	Conectarse directamente a PostgreSQL	La base de datos debe estar protegida.
Mostrar usuarios	Ejecutar SQL	SQLAlchemy pertenece al Backend.
Validar campos básicos	Guardar datos directamente	El Backend debe controlar la persistencia.
Enviar HTTP a FastAPI	Contener contraseñas de BD	La configuración sensible no debe estar en el Frontend.
Mostrar errores	Decidir reglas críticas del negocio	Las reglas importantes deben validarse en Backend.

6. Requisitos previos

- ☐ Tener Visual Studio Code instalado.
- ☐ Tener Node.js instalado.
- ☐ Tener funcionando PostgreSQL.
- ☐ Tener funcionando la API de la Guía 2.
- ☐ Poder abrir <http://127.0.0.1:8000/docs> y ejecutar los endpoints.
- ☐ Tener conocimientos básicos de HTML, CSS y JavaScript.

7. Paso 1 — Verificar Node.js

En la terminal de Visual Studio Code ejecutar:

```
node --version
```

```
npm --version
```

Si los comandos no son reconocidos, se debe instalar Node.js y reiniciar Visual Studio Code. Se recomienda una versión LTS actual.

8. Paso 2 — Crear el proyecto React con Vite

Abra Visual Studio Code. Seleccione Terminal → New Terminal y ubíquese en la carpeta donde guarda sus proyectos.

```
npm create vite@latest frontend-usuarios -- --template react
```

Después entre a la carpeta:

```
cd frontend-usuarios
```

Instale las dependencias:

```
npm install
```

Ejecute el proyecto:

```
npm run dev
```

Vite mostrará una dirección local, normalmente `http://localhost:5173`. Abra esa dirección en el navegador.

9. Paso 3 — Conocer la estructura creada

```
frontend-usuarios/  
├── public/  
├── src/  
│   ├── assets/  
│   ├── App.jsx  
│   ├── App.css  
│   ├── index.css  
│   └── main.jsx  
├── package.json  
└── vite.config.js
```

Para este ejercicio se mantendrá una estructura sencilla. Más adelante se podrán crear carpetas como `components`, `services` y `pages`.

10. Paso 4 — Crear una estructura organizada

Dentro de `src` cree:

```
src/  
├── components/  
│   ├── FormularioUsuario.jsx  
│   ├── TablaUsuarios.jsx  
│   └── Mensaje.jsx  
├── services/  
│   └── usuarioService.js  
└── App.jsx
```

```
|— App.css
|— index.css
|— main.jsx
```

Esta estructura ayuda a separar la interfaz de la comunicación con la API. No es obligatorio crear muchas carpetas en proyectos pequeños.

11. Paso 5 — Crear el servicio para FastAPI

Cree `src/services/usuarioService.js`:

```
const API_URL = "http://127.0.0.1:8000";

export async function obtenerUsuarios() {
  const respuesta = await fetch(`${API_URL}/listadeusuarios`);

  if (!respuesta.ok) {
    throw new Error("No se pudieron obtener los usuarios");
  }

  return await respuesta.json();
}

export async function crearUsuario(usuario) {
  const respuesta = await fetch(
    `${API_URL}/agregarusuarios`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(usuario)
    }
  );

  if (!respuesta.ok) {
    const error = await respuesta.json();
    throw new Error(error.detail || "No se pudo crear el usuario");
  }

  return await respuesta.json();
}

export async function eliminarUsuario(id) {
  const respuesta = await fetch(
    `${API_URL}/eliminarusuario/${id}`,
    {
      method: "DELETE"
    }
  );

  if (!respuesta.ok) {
    const error = await respuesta.json();
    throw new Error(error.detail || "No se pudo eliminar el usuario");
  }

  return await respuesta.json();
}
```

Este archivo concentra las peticiones HTTP. Así evitamos colocar todas las llamadas `fetch` dentro de `App.jsx`.

12. Paso 6 — Crear el formulario

Cree src/components/FormularioUsuario.jsx:

```
import { useState } from "react";

function FormularioUsuario({ onUsuarioCreado }) {

  const [nombre, setNombre] = useState("");
  const [apellido, setApellido] = useState("");
  const [telefono, setTelefono] = useState("");
  const [edad, setEdad] = useState("");
  const [cargando, setCargando] = useState(false);
  const [error, setError] = useState("");

  async function guardarUsuario(e) {

    e.preventDefault();
    setError("");

    if (!nombre || !apellido || !telefono || !edad) {
      setError("Todos los campos son obligatorios");
      return;
    }

    try {

      setCargando(true);

      await onUsuarioCreado({
        nombre,
        apellido,
        telefono,
        edad: Number(edad)
      });

      setNombre("");
      setApellido("");
      setTelefono("");
      setEdad("");

    } catch (error) {

      setError(error.message);

    } finally {

      setCargando(false);

    }
  }

  return (
    <form className="formulario" onSubmit={guardarUsuario}>

      <h2>Registrar usuario</h2>

      {error && (
        <div className="alerta error">
          {error}
        </div>
      )}

      <input
        type="text"
```

```

        placeholder="Nombre"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />

      <input
        type="text"
        placeholder="Apellido"
        value={apellido}
        onChange={(e) => setApellido(e.target.value)}
      />

      <input
        type="text"
        placeholder="Teléfono"
        value={telefono}
        onChange={(e) => setTelefono(e.target.value)}
      />

      <input
        type="number"
        placeholder="Edad"
        value={edad}
        onChange={(e) => setEdad(e.target.value)}
      />

      <button type="submit" disabled={cargando}>
        {cargando ? "Guardando..." : "Guardar usuario"}
      </button>

    </form>
  );
}

export default FormularioUsuario;

```

13. Paso 7 — Crear la tabla de usuarios

Cree `src/components/TablaUsuarios.jsx`:

```

function TablaUsuarios({ usuarios, onEliminar }) {

  return (
    <div className="tabla-contenedor">

      <h2>Usuarios registrados</h2>

      {usuarios.length === 0 ? (
        <p className="vacio">
          No hay usuarios registrados.
        </p>
      ) : (

        <table>

          <thead>
            <tr>
              <th>ID</th>
              <th>Nombre</th>
              <th>Apellido</th>
              <th>Teléfono</th>
              <th>Edad</th>
            </tr>

```

```

        <th>Acción</th>
      </tr>
    </thead>

    <tbody>

      {usuarios.map((usuario) => (

        <tr key={usuario.id}>

          <td>{usuario.id}</td>
          <td>{usuario.nombre}</td>
          <td>{usuario.apellido}</td>
          <td>{usuario.telefono}</td>
          <td>{usuario.edad}</td>

          <td>
            <button
              className="btn-eliminar"
              onClick={() => onEliminar(usuario.id)}
            >
              Eliminar
            </button>
          </td>

        </tr>

      )]}

    </tbody>

  </table>

)}

</div>
);
}

export default TablaUsuarios;

```

14. Paso 8 — Crear App.jsx

Reemplace el contenido de src/App.jsx por:

```

import { useEffect, useState } from "react";

import FormularioUsuario from "../components/FormularioUsuario";
import TablaUsuarios from "../components/TablaUsuarios";

import {
  obtenerUsuarios,
  crearUsuario,
  eliminarUsuario
} from "../services/usuarioService";

import "../App.css";

function App() {

  const [usuarios, setUsuarios] = useState([]);

```



```
const [cargando, setCargando] = useState(true);
const [mensaje, setMensaje] = useState("");
const [error, setError] = useState("");
```

```
async function cargarUsuarios() {

  try {

    setCargando(true);

    const datos = await obtenerUsuarios();

    setUsuarios(datos);

  } catch (error) {

    setError(error.message);

  } finally {

    setCargando(false);

  }
}
```

```
useEffect(() => {
  cargarUsuarios();
}, []);
```

```
async function manejarCrearUsuario(usuario) {

  await crearUsuario(usuario);

  setMensaje("Usuario registrado correctamente");

  await cargarUsuarios();

  setTimeout(() => {
    setMensaje("");
  }, 3000);
}
```

```
async function manejarEliminarUsuario(id) {

  const confirmar = window.confirm(
    "¿Está seguro de eliminar este usuario?"
  );

  if (!confirmar) {
    return;
  }

  try {

    await eliminarUsuario(id);

    setMensaje("Usuario eliminado correctamente");

    await cargarUsuarios();

    setTimeout(() => {
```

```

        setMensaje("");
      }, 3000);

    } catch (error) {

      setError(error.message);
    }
  }

  return (

    <div className="app">

      <header className="encabezado">

        <div className="logo-sena">
          SENA
        </div>

        <div>
          <h1>Gestión de Usuarios</h1>
          <p>Aplicación Web Full Stack</p>
        </div>

      </header>

      <main className="contenido">

        <section className="bienvenida">

          <span className="etiqueta">
            PROYECTO ACADÉMICO
          </span>

          <h2>
            Sistema de gestión de usuarios
          </h2>

          <p>
            Frontend desarrollado con React
            conectado a una API FastAPI.
          </p>

        </section>

        {mensaje && (
          <div className="alerta exito">
            {mensaje}
          </div>
        )}

        {error && (
          <div className="alerta error">
            {error}
          </div>
        )}

        <section className="tarjetas">

```

```

    <div className="tarjeta">
      <strong>{usuarios.length}</strong>
      <span>Usuarios registrados</span>
    </div>

    <div className="tarjeta">
      <strong>React</strong>
      <span>Frontend</span>
    </div>

    <div className="tarjeta">
      <strong>FastAPI</strong>
      <span>Backend</span>
    </div>

    <div className="tarjeta">
      <strong>PostgreSQL</strong>
      <span>Base de datos</span>
    </div>
  </section>

  <section className="grid">

    <FormularioUsuario
      onUsuarioCreado={manejarCrearUsuario}
    />

    <div>

      {cargando ? (
        <div className="cargando">
          Cargando usuarios...
        </div>
      ) : (

        <TablaUsuarios
          usuarios={usuarios}
          onEliminar={manejarEliminarUsuario}
        />

      )}

    </div>

  </section>

</main>

<footer>
  <p>
    Proyecto académico Full Stack
    • React + FastAPI + PostgreSQL
  </p>
</footer>

</div>
);
}

export default App;

```

15. Paso 9 — Crear el diseño visual

Reemplace `src/App.css` por el siguiente CSS. Se utiliza una paleta verde inspirada en la identidad visual del SENA:

```
:root {
  --verde-sena: #39A900;
  --verde-oscuro: #007A33;
  --verde-claro: #EAF7E3;
  --blanco: #FFFFFF;
  --gris: #F4F6F7;
  --texto: #263238;
  --rojo: #C62828;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: var(--gris);
  color: var(--texto);
}

.app {
  min-height: 100vh;
}

.encabezado {
  background: var(--verde-sena);
  color: white;
  padding: 18px 7%;
  display: flex;
  align-items: center;
  gap: 18px;
}

.logo-sena {
  background: white;
  color: var(--verde-oscuro);
  width: 60px;
  height: 60px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
  font-size: 15px;
}

.encabezado h1 {
  margin: 0;
}

.encabezado p {
  margin: 5px 0 0;
}

.contenido {
  width: 86%;
  max-width: 1250px;
}
```

```

    margin: 30px auto;
}

.bienvenida {
  background: white;
  border-radius: 18px;
  padding: 30px;
  margin-bottom: 20px;
  border-left: 7px solid var(--verde-sena);
  box-shadow: 0 5px 18px rgba(0,0,0,.08);
}

.bienvenida h2 {
  font-size: 30px;
  margin: 12px 0;
}

.etiqueta {
  color: var(--verde-oscuro);
  font-weight: bold;
  font-size: 13px;
}

.tarjetas {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
  margin-bottom: 22px;
}

.tarjeta {
  background: white;
  border-radius: 15px;
  padding: 22px;
  box-shadow: 0 4px 15px rgba(0,0,0,.07);
}

.tarjeta strong {
  display: block;
  color: var(--verde-oscuro);
  font-size: 22px;
}

.tarjeta span {
  color: #607D8B;
  font-size: 14px;
}

.grid {
  display: grid;
  grid-template-columns: 330px 1fr;
  gap: 22px;
  align-items: start;
}

.formulario,
.tabla-contenedor {
  background: white;
  border-radius: 18px;
  padding: 24px;
  box-shadow: 0 5px 18px rgba(0,0,0,.08);
}

.formulario h2,

```

```

.tabla-contenedor h2 {
  margin-top: 0;
}

.formulario input {
  width: 100%;
  padding: 12px;
  margin: 7px 0;
  border: 1px solid #CFD8DC;
  border-radius: 8px;
  font-size: 14px;
}

.formulario input:focus {
  outline: 2px solid var(--verde-sena);
}

button {
  border: none;
  border-radius: 8px;
  padding: 11px 16px;
  cursor: pointer;
  font-weight: bold;
}

.formulario button {
  width: 100%;
  margin-top: 10px;
  background: var(--verde-sena);
  color: white;
}

.formulario button:hover {
  background: var(--verde-oscuro);
}

button:disabled {
  opacity: .6;
  cursor: not-allowed;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th {
  background: var(--verde-oscuro);
  color: white;
  padding: 12px;
  text-align: left;
}

td {
  padding: 11px;
  border-bottom: 1px solid #ECEFF1;
}

tr:hover {
  background: var(--verde-claro);
}

.btn-eliminar {
  background: #FDECEC;
}

```

```

        color: var(--rojo);
    }

    .alerta {
        padding: 14px 18px;
        border-radius: 10px;
        margin-bottom: 18px;
        font-weight: bold;
    }

    .exito {
        background: #E8F5E9;
        color: var(--verde-oscuro);
    }

    .error {
        background: #FFEBEE;
        color: var(--rojo);
    }

    .cargando,
    .vacio {
        padding: 30px;
        text-align: center;
        color: #607D8B;
    }

    footer {
        margin-top: 40px;
        background: var(--verde-oscuro);
        color: white;
        text-align: center;
        padding: 20px;
    }

    @media (max-width: 900px) {

        .tarjetas {
            grid-template-columns: repeat(2, 1fr);
        }

        .grid {
            grid-template-columns: 1fr;
        }
    }

    @media (max-width: 600px) {

        .contenido {
            width: 94%;
        }

        .tarjetas {
            grid-template-columns: 1fr;
        }

        .bienvenida h2 {
            font-size: 24px;
        }

        .tabla-contenedor {
            overflow-x: auto;
        }
    }

```

16. Paso 10 — Limpiar index.css

Puede reemplazar src/index.css por:

```
html,
body,
#root {
  min-height: 100%;
}

body {
  margin: 0;
}
```

17. Paso 11 — Revisar main.jsx

Vite normalmente crea este archivo correctamente. Debe quedar similar a:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";

import App from "./App.jsx";

import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

18. Paso 12 — Verificar CORS en FastAPI

React normalmente funcionará en localhost:5173 y FastAPI en 127.0.0.1:8000. Como son orígenes diferentes, FastAPI debe permitir el origen del Frontend.

En la Guía 2 se agregó:

```
app.add_middleware(
  CORSMiddleware,
  allow_origins=[
    "http://localhost:5173",
    "http://127.0.0.1:5173"
  ],
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"]
)
```

Si React muestra un error de CORS, revise que esta configuración esté presente y que el Backend haya sido reiniciado.

19. Paso 13 — Ejecutar Backend y Frontend

Se necesitan dos terminales.

TERMINAL 1 — Backend:


```
cd ruta\del\backend  
  
venv\Scripts\activate  
  
uvicorn main:app --reload
```

TERMINAL 2 — Frontend:

```
cd frontend-usuarios  
  
npm run dev
```

Luego abra la dirección que indique Vite, normalmente <http://localhost:5173>.

20. Flujo de una operación de registro

1. El aprendiz escribe los datos en React.
2. React ejecuta `manejarCrearUsuario()`.
3. `usuarioService.js` realiza `fetch POST`.
4. FastAPI recibe la petición.
5. Pydantic valida los datos.
6. `crud.py` utiliza SQLAlchemy.
7. PostgreSQL guarda el registro.
8. FastAPI devuelve JSON.
9. React vuelve a consultar la lista.
10. La tabla se actualiza automáticamente.

Este flujo demuestra la integración completa Full Stack.

21. Pruebas funcionales obligatorias

- ☐ Abrir la página React.
- ☐ Comprobar que se muestran los usuarios existentes en PostgreSQL.
- ☐ Registrar un usuario desde el formulario.
- ☐ Comprobar que aparece inmediatamente en la tabla.
- ☐ Verificar el registro directamente en PostgreSQL.
- ☐ Eliminar un usuario desde React.
- ☐ Comprobar que desaparece de la tabla.
- ☐ Actualizar el navegador y comprobar que los datos continúan.
- ☐ Apagar y volver a iniciar el Backend y verificar que los datos continúan.
- ☐ Probar campos vacíos y observar el mensaje de validación.

22. Actividad de ampliación — Buscador

Como ejercicio, agregue un campo de búsqueda en React. Inicialmente puede realizar el filtrado en el Frontend con `filter()`. En una siguiente versión, el buscador podrá enviarse al Backend como parámetro.

```
const usuariosFiltrados = usuarios.filter((usuario) =>
  `${usuario.nombre} ${usuario.apellido}`
    .toLowerCase()
    .includes(busqueda.toLowerCase()));
```

23. Actividad de ampliación — Actualizar usuario

La Guía 2 implementa GET, POST y DELETE. Como siguiente reto, agregue PUT para actualizar un usuario. Esto permitirá completar el CRUD: Create, Read, Update y Delete.

24. Uso responsable de IA en la actividad

El aprendiz puede utilizar herramientas de IA como apoyo para comprender errores, solicitar explicaciones y generar ejemplos, pero debe ser capaz de explicar el código que entrega.

Prompt recomendado para aprender:

Actúa como profesor de React para un aprendiz de nivel intermedio.
Explicame este código paso a paso, sin cambiarlo.
Indica qué hace cada variable, función, componente y evento.
Después propón un ejercicio pequeño para comprobar que comprendí.

Prompt recomendado para solucionar un error:

Actúa como instructor de desarrollo Full Stack.
Tengo un proyecto React conectado a FastAPI.
Este es el error:
[PEGAR ERROR]

Este es el código relacionado:
[PEGAR CÓDIGO]

Explicame primero la causa del error y después dame una solución sencilla explicando cada cambio.

No se debe entregar código generado por IA que el aprendiz no comprenda. La evidencia debe demostrar aprendizaje.

25. Problemas frecuentes y soluciones

Failed to fetch

Verifique que FastAPI esté ejecutándose en el puerto 8000 y que la URL del servicio sea correcta.

CORS policy

Revise CORSMiddleware y que el origen de React coincida con localhost:5173.

404 en /listadeusuarios

Compruebe que el endpoint exista exactamente en FastAPI.

POST devuelve error 422

Revise los nombres y tipos de los campos enviados por React.

No aparecen usuarios

Compruebe PostgreSQL, el Backend y GET /listadeusuarios en Swagger.

La página React no inicia

Ejecute npm install y después npm run dev.

Cambios no aparecen

Guarde los archivos y revise la consola del navegador y la terminal.

26. Evidencias de aprendizaje

- Captura de la estructura del proyecto React en Visual Studio Code.
- Captura de npm run dev funcionando.
- Captura de la página web terminada.
- Captura del formulario de registro.
- Captura de la tabla de usuarios.
- Captura de un usuario creado desde React.
- Captura del registro correspondiente en PostgreSQL.
- Captura de la eliminación desde React.
- Captura de Swagger mostrando el Backend.
- Explicación del recorrido de los datos desde React hasta PostgreSQL.

27. Criterios de evaluación sugeridos

Criterio	Valor	Descripción
Creación y configuración React	15%	Proyecto Vite funcionando correctamente.
Consumo de API	25%	GET, POST y DELETE funcionando desde React.
Componentes y Hooks	20%	Uso correcto de componentes, useState y useEffect.
Interfaz y experiencia de usuario	20%	Diseño moderno, responsive y claro.
Integración Full Stack	15%	React → FastAPI → SQLAlchemy → PostgreSQL.
Buenas prácticas	5%	Código organizado, legible y comprendido.

28. Reto final

Transforme la aplicación en una pequeña plataforma académica. Mantenga el diseño desarrollado y agregue una sección de navegación, búsqueda de usuarios, actualización de usuarios y un contador de registros.

- Agregar botón Editar.
- Crear formulario de actualización.

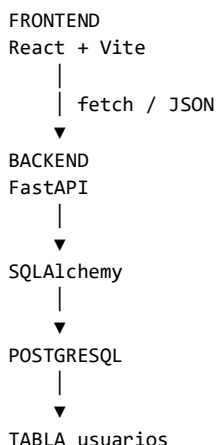
- Implementar PUT en FastAPI.
- Crear updateUsuario() en usuarioService.js.
- Agregar buscador por nombre o apellido.
- Mostrar un contador de usuarios.
- Mejorar la experiencia en dispositivos móviles.
- Agregar un mensaje de confirmación antes de eliminar.

29. Reflexión final

Responda con sus propias palabras: ¿qué función cumple React en el proyecto? ¿Por qué React no debe conectarse directamente a PostgreSQL? ¿Qué función cumple fetch? ¿Qué hacen useState y useEffect? ¿Qué problema soluciona CORS? ¿Qué ocurre desde que el usuario presiona 'Guardar usuario' hasta que el registro aparece en PostgreSQL?

30. Resultado esperado

Al finalizar la guía, el aprendiz tendrá una aplicación web funcional y visualmente organizada, capaz de comunicarse con el Backend construido en la Guía 2.



31. Lista de comprobación final

- ☐ React instalado y ejecutándose.
- ☐ FastAPI ejecutándose.
- ☐ PostgreSQL ejecutándose.
- ☐ CORS configurado.
- ☐ GET funciona.
- ☐ POST funciona.

- ☐ DELETE funciona.
- ☐ Los datos permanecen en PostgreSQL.
- ☐ La interfaz tiene diseño responsive.
- ☐ El aprendiz puede explicar el flujo completo.